



南開大學
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

MPI 编程实验——ANN 选题

王 烨

年级：2024 级

专业：计算机科学与技术

指导教师：王刚

2026 年 6 月 5 日

目录

一、 实验目的	1
二、 实验环境	1
三、 实验代码	1
四、 实验流程	1
(一) 关于 MPI 的基本介绍	1
(二) IVF-SIMD MPI	1
1. IVF-SIMD 单线程 MPI	1
trade-off 测试	3
2. IVF-SIMD 多线程 MPI	4
trade-off 测试	5
OMP 对负载均衡的影响	7
np 对负载均衡的影响	7
Nodes 和 ppn 对负载均衡的影响	7
多线程与多进程的负载均衡	7
(三) HNSW MPI	9
1. IVF+HNSW MPI	9
trade-off 测试	11
2. HNSW Partition MPI	12
trade-off 测试	12
3. HNSW ON HNSW MPI	14
trade-off 测试	15
五、 总结	17

一、 实验目的

- 探索在 MPI 多进程上的数据划分和并行搜索。
- 针对 ANNS 算法（如 IVF、图索引）实现多进程并行搜索，探索数据划分与搜索策略。
- 探索 MPI 与多线程/SIMD 混合并行及通信计算重叠，优化负载均衡与 Latency-Recall 权衡。

二、 实验环境

与多线程实验一致，本实验基于华为鲲鹏服务器开展，使用 ARM arrch64 架构和 128 位 NEON 指令集，通用编译指令为 `mpic++ main.cc -o main -O2 -fopenmp -std=c++11 -lpthread`，在服务器端的测试通过脚本 `qsub_mpi.sh` 开展。

三、 实验代码

本人实验的所有代码，实验结果，包括但不限于可视化和算法脚本都可以在 <https://github.com/wangyeG2/Parallel-Programming-NKU-2026-Spring.git> 中查看。

四、 实验流程

（一） 关于 MPI 的基本介绍

消息传递接口（Message Passing Interface, MPI），是基于消息传递的一种并行编程规范和编程环境，实现并行执行的进程或线程之间的数据交换和同步。MPI 提供了多种通信模式，包括点对点通信（阻塞与非阻塞）、组通信（集合操作，collective operations）等，以实现高效的进程间数据交换与协同。

在 MPI 的语境中，进程特指独立使用 MPI 进行通信的一个个体。一个 MPI 并行计算应用程序则由一组运行在相同或不同计算节点上的进程组成。在 Linux OS 上，一个进程就是一个 Unix 进程，它使用 MPI 进行通信。同一个 MPI 进程只能运行在一个节点上，一个节点上可以运行若干个 MPI 进程，并且这些进程运行着相同的代码逻辑和处理不同的数据（Single Program Multiple Data）。

（二） IVF-SIMD MPI

在原先的多线程编程实验中，我们探索了 IVF（倒排文件索引）对于 ANN 的优化，开展了 IVF-SIMD 优化和 IVF-SIMD + OpenMP 多线程优化等探索。在本次实验中，我们引入 MPI，对上述算法进行进一步探索。

1. IVF-SIMD 多线程 MPI

我们构建了新的 `ivf-simd-mpi.h`，在原先 IVF-SIMD 无多线程优化版本的基础上，Rank 0 进程执行全局 K-Means 聚类计算出全局质心，然后通过 `MPI_Bcast` 广播给所有进程。各进程根据分块处理自己负责的 Base 数据，独立构建倒排列表并保存全局 ID。搜索过程中，粗排阶段各进程独立利用 NEON 指令计算 query 与全局质心的内积，找出 `nprobe` 个最近簇，然后在精排阶段各自维护自己的倒排索引表，最后执行全局合并，将局部结果转换为定长数组。

Algorithm 1 构建 IVF 索引 (MPI 分布式)**Input:** 基础数据集 B , 向量数量 N , 维度 D , 聚类中心数 $nlist$, 进程号 $rank$, 总进程数 $size$ **Output:** IVF 索引 (聚类中心 C , 本地倒排列表 $InvLists$)

```

1: if  $rank = 0$  then
2:   从  $B$  中随机选择  $nlist$  个向量初始化聚类中心  $C$ 
3:   for  $iter = 1$  到  $10$  do
4:     for 每个向量  $x_i \in B$  do
5:        $c_i \leftarrow \arg \max_c \text{ComputeIP}_{\text{NEON}}(x_i, C_c)$  ▷ 分配到最近聚类中心
6:     end for
7:     for 每个聚类中心  $c \in \{1, \dots, nlist\}$  do
8:        $C_c \leftarrow \frac{1}{|S_c|} \sum_{x_i \in S_c} x_i$  ▷ 对分配的向量求平均以更新聚类中心
9:     end for
10:   end for
11: end if
12: 通过 MPI_Bcast 将聚类中心  $C$  广播给所有进程
13: 计算数据块大小  $chunk \leftarrow \lceil N/size \rceil$ , 起始索引  $start \leftarrow rank \times chunk$ , 结束索引  $end \leftarrow \min(start + chunk, N)$ 
14: for 每个向量  $x_i$ , 其中  $i \in [start, end)$  do
15:    $c \leftarrow \arg \max_c \text{ComputeIP}_{\text{NEON}}(x_i, C_c)$  ▷ 寻找最近聚类中心
16:    $InvLists[c].\text{push\_back}(i)$  ▷ 将全局 ID 插入本地倒排列表
17: end for
18: return IVF 索引

```

Algorithm 2 搜索 IVF 索引 (MPI 分布式)**Input:** IVF 索引, 查询向量 q , 基础数据集 B , Top-K 值 k , 探测聚类数 $nprobe$, 进程号 $rank$, 总进程数 $size$ **Output:** 全局 Top-K 结果

```

1: for 每个聚类中心  $c \in \{1, \dots, nlist\}$  do
2:    $dist_c \leftarrow \text{ComputeIP}_{\text{NEON}}(q, C_c)$  ▷ ARM NEON 优化的内积计算
3: end for
4:  $C_{top} \leftarrow$  内积最大的前  $nprobe$  个聚类中心 ▷ 粗排
5: 初始化最大堆  $PQ_{local}$ , 容量为  $k$ , 存储 (负内积, 向量 ID)
6: for 每个聚类中心  $c \in C_{top}$  do
7:   for 每个向量 ID  $i \in InvLists[c]$  do
8:      $ip \leftarrow \text{ComputeIP}_{\text{NEON}}(q, B[i])$  ▷ 精排
9:      $neg\_ip \leftarrow -ip$ 
10:    if  $|PQ_{local}| < k$  then
11:       $PQ_{local}.\text{push}((neg\_ip, i))$ 
12:    else if  $neg\_ip < PQ_{local}.\text{top}().neg\_ip$  then
13:       $PQ_{local}.\text{pop}()$ 
14:       $PQ_{local}.\text{push}((neg\_ip, i))$ 
15:    end if
16:   end for
17: end for

```

```

18: 将  $PQ_{local}$  转换为定长数组  $A_{local}$  (大小为  $k$ ), 空余位置用  $(0, \infty)$  补齐
19: 通过 MPI_Gather 将所有  $A_{local}$  收集到 Rank 0 的  $A_{global}$  中
20: if  $rank = 0$  then
21:   初始化最大堆  $PQ_{global}$ , 容量为  $k$ , 存储 (负内积, 向量 ID)
22:   for 每个元素  $e \in A_{global}$  do
23:     if  $e.id \neq \infty$  then ▷ 过滤无效的补齐数据
24:       if  $|PQ_{global}| < k$  then
25:          $PQ_{global}.push(e)$ 
26:       else if  $e.neg\_ip < PQ_{global}.top().neg\_ip$  then
27:          $PQ_{global}.pop()$ 
28:          $PQ_{global}.push(e)$ 
29:       end if
30:     end if
31:   end for
32:   return  $PQ_{global}$ 
33: end if

```

trade-off 测试 在 MPI 编程实验中, 关于 recall 和 latency 的 trade-off 测试仍然是关键讨论之一。针对 ivf-simd-mpi.h, 由于我们暂时没有引入多线程优化, 关键参数因子为: 搜索簇大小 $nprobe$ 和申请核心数 ppn , 我们规定节点数 $nodes=1$, $np=nodes \times ppn$ 。对于测试数据的整理如下¹:

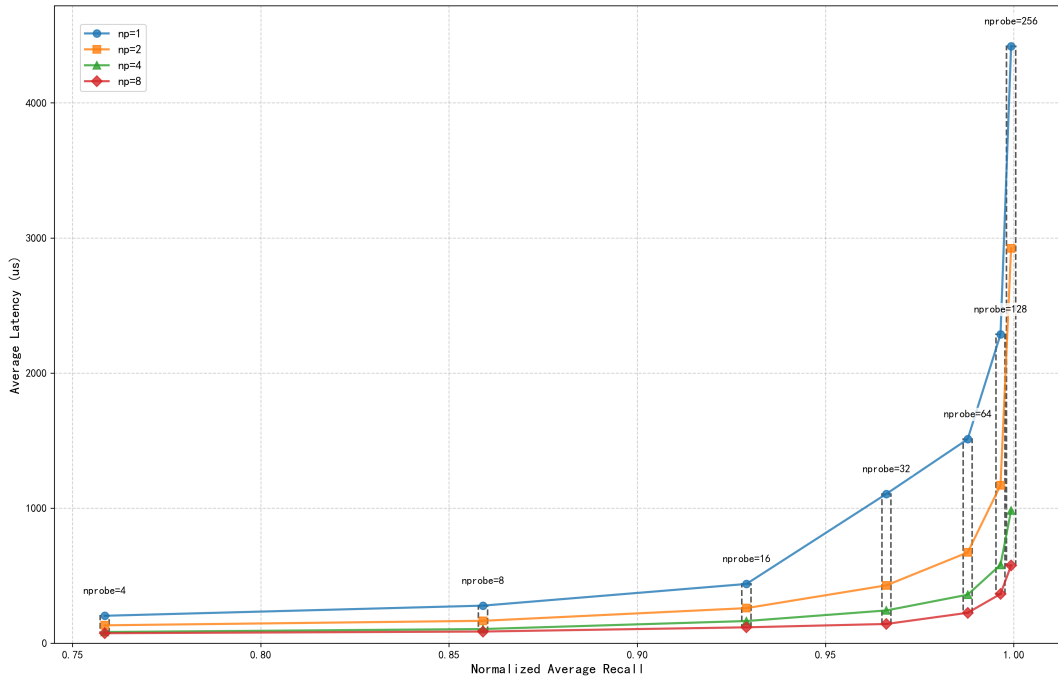


图 1: IVF-SIMD-MPI trade-off

¹在折线图绘制过程中, 对于相同的 $nprobe$, 其 recall 差异基本来源于浮点数计算误差, 为了保证绘图的规整和美观, 我们对相同 $nprobe$ 取值的数据点, recall 采取归一化操作, 统一取其平均值。

表 1: IVF-SIMD-MPI 算法综合性能对比（最优值加粗）

nprobe	np	recall	延迟 (us)	加速比
4	1	0.76115	203.333	1.00×
8	1	0.86140	278.424	0.73×
16	1	0.92870	438.884	0.46×
32	1	0.96640	1105.36	0.18×
64	1	0.98745	1510.28	0.13×
128	1	0.99680	2286.34	0.09×
256	1	0.99900	4418.88	0.05×
4	2	0.75635	132.755	1.53×
8	2	0.85980	166.1	1.22×
16	2	0.92905	259.971	0.78×
32	2	0.96665	428.552	0.47×
64	2	0.98835	672.106	0.30×
128	2	0.99570	1169.18	0.17×
256	2	0.99930	2922.31	0.07×
4	4	0.75725	83.167	2.44×
8	4	0.85970	105.239	1.93×
16	4	0.93005	164.429	1.24×
32	4	0.96580	242.803	0.84×
64	4	0.98685	359.284	0.57×
128	4	0.99650	580.571	0.35×
256	4	0.99935	982.154	0.21×
4	8	0.75945	74.234	2.74×
8	8	0.85525	86.703	2.35×
16	8	0.92820	117.899	1.72×
32	8	0.96580	142.695	1.42×
64	8	0.98840	225.422	0.90×
128	8	0.99695	364.931	0.56×
256	8	0.99955	575.904	0.35×

从折线图中，我们可以明显看出，随着 nprobe 增大，latency 和 recall 均增大，这和我们多线程编程实验中的理论分析一致：nprobe 越大，包含真实最近邻的概率就越大，recall 便会上升，同时参与精确计算的基础向量数量成比例增加，计算量增大，latency 也会上升。此外，随着进程数 np 增大，计算量被等量分配到各进程中，且在单节点内通信开销极小，latency 基本呈现线性降低的趋势，且 recall 基本保持不败你，这说明了我们的 MPI 实验在保证算法正确性的同时有效优化了算法性能。

2. IVF-SIMD 多线程 MPI

上述的 IVF-SIMD-MPI 并没有启用多线程优化手段，理论上仍然存在优化空间。在精排阶段，我们引入 OpenMP 多线程优化，为每个线程维护独立的 Top-K 结果堆，同时，针对倒排列

表长度不一导致的各簇计算量差异, 使用动态调度 (schedule(dynamic)) 来分配簇级任务, 有效解决了负载不均衡问题。各线程完成本地倒排列表遍历后, 再将线程局部结果无锁合并至进程级队列中, 最后由 MPI 完成跨节点的全局 Top-K 归约。

Algorithm 3 搜索 IVF 索引 (MPI + OpenMP 多线程)

Input: IVF 索引, 查询向量 q , 基础数据集 B , Top-K 值 k , 探测聚类数 $nprobe$, 进程号 $rank$, 总进程数 $size$

Output: 全局 Top-K 结果

```

1: for 每个聚类中心  $c \in \{1, \dots, nlist\}$  do
2:    $dist_c \leftarrow \text{ComputeIP}_{\text{NEON}}(q, C_c)$  ▷ 粗排: 计算查询与质心内积
3: end for
4:  $C_{top} \leftarrow$  内积最大的前  $nprobe$  个聚类中心
5: 初始化进程级最大堆  $PQ_{local}$ , 容量为  $k$ , 存储 (负内积, 向量 ID)
6: // 精排: OpenMP 簇级多线程并行
7: parallel region begin ▷ 启动 OpenMP 并行区域
8: 初始化线程级最大堆  $PQ_{thread}$ , 容量为  $k$ 
9: for  $i = 0$  to  $nprobe - 1$  do (schedule(dynamic), nowait) ▷ 动态调度解决负载不均衡
10: 获取聚类中心  $c \leftarrow C_{top}[i]$  及其倒排列表  $InvLists[c]$ 
11: for 每个向量 ID  $id \in InvLists[c]$  do
12:    $ip \leftarrow \text{ComputeIP}_{\text{NEON}}(q, B[id])$ 
13:   维护  $PQ_{thread}$  插入或替换  $(-ip, id)$ 
14: end for
15: end for
16: critical region begin ▷ 临界区: 合并线程结果
17: while  $PQ_{thread}$  不为空 do
18:   弹出元素  $e \leftarrow PQ_{thread}.top()$ 
19:   维护  $PQ_{local}$  插入或替换  $e$ 
20: end while
21: critical region end
22: parallel region end
23: 将  $PQ_{local}$  转换为定长数组  $A_{local}$  (大小为  $k$ ), 空余位置用  $(0, \infty)$  补齐
24: 通过 MPI_Gather 将所有  $A_{local}$  收集到 Rank 0 的  $A_{global}$  中
25: if  $rank = 0$  then
26:   初始化全局最大堆  $PQ_{global}$ , 容量为  $k$ 
27:   for 每个元素  $e \in A_{global}$  do
28:     if  $e.id \neq \infty$  then ▷ 过滤无效的补齐数据
29:       维护  $PQ_{global}$  插入或替换  $e$ 
30:     end if
31:   end for
32:   return  $PQ_{global}$ 
33: end if

```

trade-off 测试 我们在 IVF-SIMD 中同时引入了多线程和多进程优化, 似乎起到了双重优化的效果。然而, 在并行计算任务中, 通信开销是不可忽略的关键因素, 线程数越多, 各部分的计算量降低, 但是相应的调度开销和 cache miss 也会增大, 后者在线程数达到一定规模时会对算法

性能起到严重的负优化，这也是我们在本阶段实验所要进行的 trade-off 测试。

在本阶段测试中，由于 nprobe 的影响已经在前文被再次阐述证明，我们不再对这一参数进行调节，全程控制 nprobe=16，此时关键参数因子为：节点数 nodes，申请核心数 ppn 和线程数 OMP。我们规定节点数 $nodes \leq 4$, $ppn \leq 8$ ，参数之间满足约束： $np \times OMP = nodes \times ppn$ 。

由于我们控制了 nprobe 的取值，recall 基本不变，细微差距来源于浮点计算误差，在理论分析意义上忽略不计。所以本阶段 trade-off 的本质是“计算加速”与“通信开销”的负载均衡博弈²。

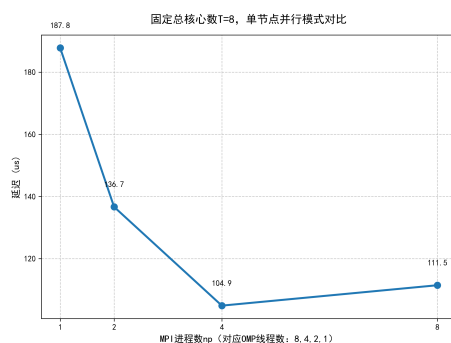


图 2: 单节点下 np/OMP 比例的影响

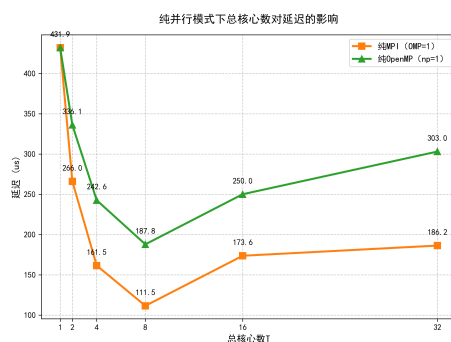


图 3: 纯并行模式下总核心数的影响

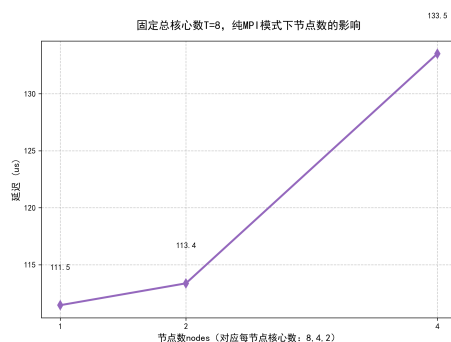


图 4: 跨节点通信对延迟的影响

²狭义上我们认为负载均衡 (Load Balance) 指将负载 (工作任务) 进行平衡、分摊到多个操作单元上进行运行，实现最大化资源利用率、减少空闲时间并避免单点过载。此处我们对于负载均衡的语义进行扩大，是一种让计算、通信、同步、内存访问等所有系统资源的开销达到全局最优的平衡状态。

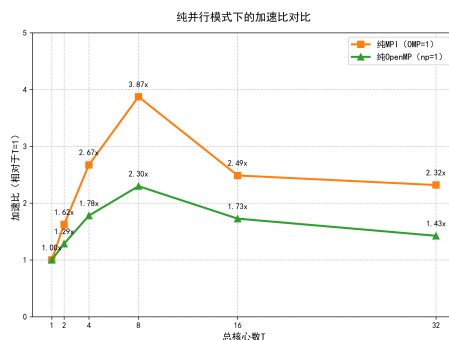


图 5: 纯并行模式下的加速比对比

OMP 对负载均衡的影响 我们在精排阶段引入了 OpenMP 多线程优化，增加 OMP 可以并行化簇的遍历，解决负载均衡，但随之带来的是临界区锁竞争和内存带宽争用。从数据中可以看出，单节点 (nodes=1) 下纯 MPI (OMP=1,ppn=8) 算法性能显著高于纯 OpenMP (OMP=8,ppn=1) 算法，这是因为 OMP=8 时，多个线程共享内存且频繁争抢临界区，导致了 cache miss 和带宽挤兑，本质是达到了 memory-bound。

np 对负载均衡的影响 增加 np 可以将 Base 数据切分到更多进程，减少单进程计算量，但同时 MPI_Bcast 和 MPI_Gather 的通信量线性增加，Rank 0 进程的 merge 开销也随之增加。从数据中可以看出，单节点 (nodes=1) 下随着 np 增大，latency 显著降低，而多节点通信下，np 增大带来了巨大的通信开销，Nodes=4, ppn=8, np=32, OMP=1 耗时 186.2us，是 Nodes=4, ppn=2, np=8, OMP=2 的 94.6us 的 2 倍。这是由于 32 个进程跨节点进行 MPI_Gather 时，网络延迟和 Rank 0 的串行 merge 开销占据了主导地位。

Nodes 和 ppn 对负载均衡的影响 Nodes 决定了执行计算任务的节点数，ppn 决定了单节点内最多可执行的进程数，两者之间存在显著的通信开销与计算开销的平衡。当 np=8 时，增大 nodes 并没有带来计算延迟的降低，反而略有上升，这是节点间增加的通信开销的超过了多节点降低的计算开销。此外，注意到在测试数据中，在 Nodes=4, ppn=8, np=8, OMP=4 等测试点出现了严重的延迟上升，达到了 1374.5us，这是由于节点内内存带宽被 8 个线程打满，节点间网络通信又被阻塞，计算开销和通信开销出现了双重叠加，导致了严重的性能降低。

多线程与多进程的负载均衡 从我们的测试结果可以看出，在总核心数 T (ppn*nodes) 不变的前提下，纯 MPI (OMP=1) 算法的 latency 始终低于纯 OpenMP (np=1) 算法，这说明在 ANN 计算任务中，纯 MPI 的效果优于纯 OpenMP，同时注意到，当 np 的占比更高时，对于 latency 的降低效果更优，最优 latency 测试点分布在 OMP=2 的情况下，最优测试参数为 nodes=4, ppn=4, np=8, OMP=2, latency 为 94.633us。

正如前文所述，如果 OMP 过大，会带来锁竞争和内存带宽争用，面对的问题是 memory-bound，而当 OMP=2 时，可以同时发挥分摊计算量和平衡负载均衡的效果，而在此基础上，适当增大 nodes 可以分片降低单节点负载，再配合适宜的 np 取值，可以有效降低单进程计算量。由此可以得出，OpenMP 和 MPI 混行是 ANN 计算任务的最优算法。

完整的测试数据表格如下：

表 2: IVF-SIMD-MPI-多线程算法综合性能对比 (最优值加粗)

nodes	ppn	np	OMP	recall	延迟 (us)	加速比
1	1	1	1	0.927303	431.866	1.00×
1	2	1	2	0.926902	336.053	1.29×
1	2	2	1	0.925202	265.970	1.62×
1	4	1	4	0.929552	242.574	1.78×
1	4	2	2	0.926002	192.514	2.24×
1	4	4	1	0.926652	161.476	2.67×
1	8	1	8	0.927753	187.779	2.30×
1	8	2	4	0.925703	136.658	3.16×
1	8	4	2	0.927953	104.893	4.12×
1	8	8	1	0.926203	111.453	3.88×
2	1	1	2	0.928552	319.253	1.35×
2	1	2	1	0.925652	294.074	1.47×
2	2	1	4	0.926703	218.935	1.97×
2	2	2	2	0.926702	210.495	2.05×
2	2	4	1	0.926403	162.144	2.66×
2	4	1	8	0.927652	198.464	2.18×
2	4	2	4	0.927552	166.217	2.60×
2	4	4	2	0.925653	129.236	3.34×
2	4	8	1	0.930503	113.370	3.81×
2	8	1	16	0.924652	250.012	1.73×
2	8	2	8	0.924653	576.323	0.75×
2	8	4	4	0.926253	590.763	0.73×
2	8	8	2	0.928852	540.394	0.80×
2	8	16	1	0.928253	173.594	2.49×
4	1	1	4	0.928253	173.594	2.49×
4	1	2	2	0.925602	144.554	2.99×
4	1	4	1	0.925853	176.355	2.45×
4	2	1	8	0.925853	176.355	2.45×
4	2	2	4	0.925753	139.482	3.10×
4	2	4	2	0.923902	100.382	4.30×
4	2	8	1	0.925852	133.517	3.24×
4	4	1	16	0.927053	221.809	1.95×
4	4	2	8	0.926702	631.031	0.68×
4	4	4	4	0.924453	563.875	0.77×
4	4	8	2	0.928702	94.633	4.56×
4	4	16	1	0.924103	123.968	3.48×
4	8	1	32	0.927553	302.983	1.43×
4	8	2	16	0.926403	381.622	1.13×
4	8	4	8	0.925003	1578.180	0.27×
4	8	8	4	0.928003	1374.550	0.31×
4	8	16	2	0.928952	700.263	0.62×
4	8	32	1	0.924703	186.214	2.32×

(三) HNSW MPI

在多线程编程实验中，我们探索了基于图索引的 ANN 算法优化，基于开源库 hnsplib 设计了相应的 HNSW 算法，并做出了 IVF+HNSW 的并行优化尝试，效果尚可接受。在本实验中，我们基于现有的 HNSW 图索引算法，探索 MPI 对于算法的并行优化可能。

1. IVF+HNSW MPI

首先通过 KMeans 聚类将高维向量数据划分为多个簇，每个簇构建独立的 HNSW 索引。构建阶段中，先训练 KMeans 得到簇质心，然后将每个数据点分配到最近的簇，对每个非空簇分别构建 HNSW 索引并保存。查询阶段时，先计算查询向量与所有簇质心的距离，选取距离最近的 $nprobe$ 个簇；然后在每个选中簇的 HNSW 索引中进行精确搜索；最后将各簇的搜索结果合并，通过排序得到最终的 top-k 结果。这是我们 IVF+HNSW 的算法流程。为什么可以使用 MPI 优化？正如前文对于 IVF-SIMD 的 MPI 探索一样，我们可以在 KMeans 训练阶段通过 Rank 0 进程计算全局质心并广播给所有进程，各自分块处理数据并建立倒排索引表，最后全局合并，这一阶段与 HNSW 处理互相独立，所以启用 MPI 优化在算法正确性上是没有问题的。

我们设计了新的 ivf-hnsw-mpi.h，使用与 IVF-SIMD MPI 相似的手段进行优化：

Algorithm 4 IVF-HNSW-MPI 索引构建

Input: 数据集 $\mathcal{B} = \{\mathbf{x}_i\}_{i=1}^n$, 维度 d , 簇数 $nlist$, HNSW 参数 $M, ef_construction$, MPI 进程号 $rank$, 进程总数 $size$

Output: 磁盘上保存的索引文件

- 1: 创建索引目录 (进程 0 执行, 之后同步所有进程)
 - 2: **if** $rank = 0$ **then**
 - 3: 对 $\mathcal{B}$ 执行 KMeans 聚类, 得到聚类中心 $\mathbf{C} = \{\mathbf{c}_j\}_{j=1}^{nlist}$
 - 4: 将 $\mathbf{C}$ 保存至文件 `centroids.bin`
 - 5: **end if**
 - 6: 广播 $\mathbf{C}$ 至所有进程 ▷ MPI_Bcast
 - 7: 计算本进程负责的簇范围:

$$clusters_per_rank \leftarrow \lfloor nlist/size \rfloor, \quad remainder \leftarrow nlist \bmod size$$

$$start_cluster \leftarrow rank \times clusters_per_rank + \min(rank, remainder)$$

$$end_cluster \leftarrow start_cluster + clusters_per_rank + \mathbb{I}[rank < remainder]$$
 - 8: **for** 每个向量 $\mathbf{x}_i \in \mathcal{B}$ **do**
 - 9: $best_cluster \leftarrow \arg \min_j IPDist(\mathbf{x}_i, \mathbf{c}_j)$ ▷ 内积距离
 - 10: 将 $\mathbf{x}_i$ 及其 ID 分配至簇 $best_cluster$
 - 11: **end for**
 - 12: **for** 每个本进程负责的簇 $c \in [start_cluster, end_cluster)$ **do**
 - 13: **if** 簇 c 非空 **then**
 - 14: 创建 HNSW 索引 H_c , 参数为 $M, ef_construction$
 - 15: 将簇 c 中所有向量并行插入 H_c ▷ OpenMP 并行
 - 16: 保存 H_c 至文件 `cluster_c.bin`
 - 17: **end if**
 - 18: **end for**
 - 19: 同步所有进程 ▷ MPI_Barrier
-

Algorithm 5 IVF-HNSW-MPI 索引加载

Input: 索引路径 $path$, MPI 进程号 $rank$, 进程总数 $size$

Output: 各进程加载其负责的簇索引

```

1: if  $rank = 0$  then
2:   从文件读取聚类中心  $C$ 
3: end if
4: 广播  $C$  至所有进程                                ▷ MPI_Bcast
5: 计算本进程负责的簇范围  $[start\_cluster, end\_cluster)$     ▷ 同构建时的划分方式
6: for 每个簇  $c \in [start\_cluster, end\_cluster)$  do
7:   if 文件  $cluster\_c.bin$  存在 then
8:     从磁盘加载 HNSW 索引  $H_c$ 
9:   end if
10: end for

```

Algorithm 6 IVF-HNSW-MPI 查询搜索

Input: 查询向量 q , 返回数 k , 探测簇数 $nprobe$, 搜索参数 ef_search

Output: 全局 Top- k 最近邻结果 $\mathcal{R}$

阶段 1: 确定探测簇

```

1: 每个进程计算  $q$  到本进程负责的各簇中心的内积距离:
    $D_{local} \leftarrow \{(\text{IPDist}(q, c_j), j) \mid j \in [start\_cluster, end\_cluster)\}$ 
2: 收集所有进程的  $D_{local}$  至进程 0                                ▷ MPI_Gatherv
3: if  $rank = 0$  then
4:    $D_{global} \leftarrow$  合并所有进程的距离对
5:   对  $D_{global}$  按距离升序部分排序, 取前  $nprobe$  个簇
6:    $P \leftarrow$  选出的  $nprobe$  个簇 ID 集合
7: end if
8: 广播  $P$  至所有进程                                ▷ MPI_Bcast

```

阶段 2: 局部搜索

```

9:  $\mathcal{R}_{local} \leftarrow \emptyset$ 
10: for 每个探测簇  $c \in P$  do
11:   if  $c \in [start\_cluster, end\_cluster)$  且  $H_c \neq \emptyset$  then
12:     设置  $H_c.ef \leftarrow ef\_search$ 
13:      $\mathcal{R}_c \leftarrow H_c.\text{searchKnn}(q, k)$ 
14:      $\mathcal{R}_{local} \leftarrow \mathcal{R}_{local} \cup \mathcal{R}_c$ 
15:   end if
16: end for

```

阶段 3: 全局结果合并

```

17: 收集所有进程的  $\mathcal{R}_{local}$  至进程 0                                ▷ MPI_Gatherv
18: if  $rank = 0$  then
19:    $\mathcal{R}_{global} \leftarrow$  合并所有进程的局部搜索结果
20:   对  $\mathcal{R}_{global}$  按距离升序部分排序, 取前  $k$  个
21:    $\mathcal{R} \leftarrow$  全局 Top- $k$  结果
22: end if
23: return  $\mathcal{R}$                                 ▷ 仅进程 0 拥有完整结果

```

trade-off 测试 与先前的 trade-off 测试一样，本阶段关键参数因子为：节点数 $nodes$ 和申请核心数 ppn ， $np=nodes \times ppn$ 。

表 3: IVF-HNSW MPI 优化算法综合性能对比（最优值加粗）

nodes	ppn	np	recall	延迟 (us)	加速比
1	1	1	0.918302	978.149	1.00×
1	2	2	0.918302	740.835	1.32×
1	4	4	0.918302	367.761	2.66×
1	8	8	0.918302	259.691	3.77×
2	1	2	0.918302	747.803	1.31×
2	2	4	0.918302	468.195	2.09×
2	4	8	0.918302	432.864	2.26×
2	8	16	0.918302	496.418	1.97×
4	1	4	0.918302	554.058	1.77×
4	2	8	0.918302	463.179	2.11×
4	4	16	0.918302	480.703	2.03×
4	8	32	0.918302	785.626	1.24×

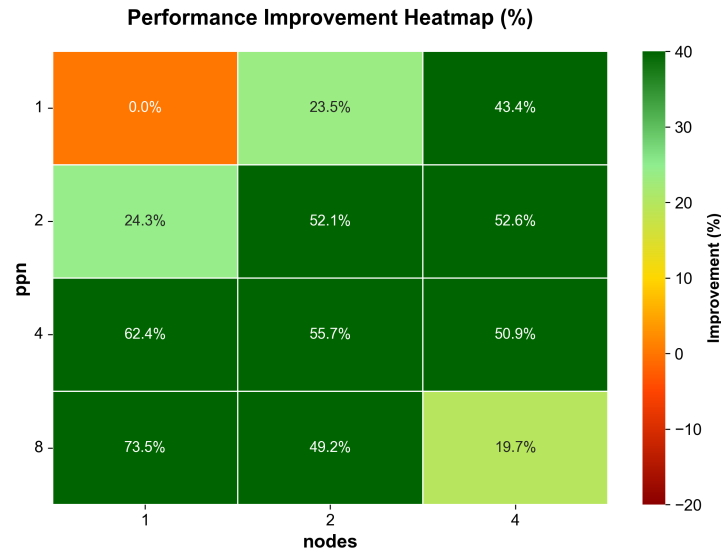


图 6: IVF-HNSW-MPI performance_heatmap

与先前对于 IVF-SIMD-MPI 多线程优化的分析结果不同，针对 IVF-HNSW-MPI 的测试结果，出现了以下变化：

- 通信开销的影响显著增大：当 ppn 保持在较高水平时，增大节点数 $nodes$ ，延迟显著上升，这是由于 IVF-HNSW-MPI 需先通过 `MPI_Gatherv` 收集所有进程的簇中心距离（用于粗排），再收集每个簇的 HNSW 搜索结果。此外，HNSW 的图遍历具有强数据依赖性，进程内 OpenMP 并行的收益有限（线程间路径探索差异大，负载不均），更多依赖 MPI 进程间并行，跨节点通信延迟成为延迟的主要瓶颈。
- 进程数对于性能的影响效果提升：IVF-HNSW-MPI 的计算量扩大，使进程数对于计算开

销的影响变得更加显著，在单节点（nodes=1）的情况下，随着 ppn 的增大，加速比基本呈现线性增长的趋势。

2. HNSW Partition MPI

上述算法是对 HNSW 的一种 MPI 优化尝试。事实上，针对 HNSW，与 IVF-SIMD 中倒排列表长度不均导致的负载均衡问题不同，HNSW 主要受限于簇的数据量不均，簇的划分不均，会导致 HNSW 索引过大，搜索时间增加，会出现进程间负载不均。因此，探索对簇的不同划分方式，是对于 HNSW 算法性能优化新的尝试。

我们设计了新的 partition_hnsw_mpi.h，直接将数据集划分为 P 个部分，然后对每个部分单独建立 HNSW，搜索时利用 MPI 在 HNSW 间进行并行搜索，最后进行 top-k 的 merge：

Algorithm 7 Partition-HNSW-MPI 索引构建

Input: 基数据集 $\mathcal{B} = \{\mathbf{x}_i\}_{i=0}^{n-1}$, 维度 d , HNSW 参数 M , $ef_construction$, MPI 进程号 $rank$, 进程总数 $size$

Output: 磁盘上保存的分区索引文件

- 1: 创建索引目录（进程 0 执行，之后同步所有进程）
 - 2: 计算本进程负责的数据分片范围（均分策略）：

$$base_n \leftarrow \lfloor n/size \rfloor, \quad remainder \leftarrow n \bmod size$$

$$local_n \leftarrow base_n + \mathbb{I}[rank < remainder]$$

$$local_offset \leftarrow rank \times base_n + \min(rank, remainder)$$
 - 3: 拷贝本地的数据分片 $\mathcal{B}_{local} \leftarrow \mathcal{B}[local_offset : local_offset + local_n]$
 - 4: 初始化本地 HNSW 索引 H_{local} ，参数为 $M, ef_construction$
 - 5: **if** $local_n > 0$ **then**
 - 6: 插入第一个点: $H_{local}.addPoint(\mathcal{B}_{local}[0], local_offset)$ ▷ ID 设为全局偏移量
 - 7: **for** $i = 1$ **to** $local_n - 1$ **do** ▷ OpenMP 并行插入
 - 8: $H_{local}.addPoint(\mathcal{B}_{local}[i], local_offset + i)$ ▷ ID 设为全局偏移量
 - 9: **end for**
 - 10: **end if**
 - 11: 保存 H_{local} 至文件 `partition_rank.bin`
 - 12: 同步所有进程 ▷ MPI_Barrier
-

Algorithm 8 Partition-HNSW-MPI 索引加载

Input: 索引路径 $path$, 数据总量 n , MPI 进程号 $rank$, 进程总数 $size$

Output: 各进程加载其负责的分区索引

- 1: 计算本进程负责的数据分片范围 $local_n$ 和 $local_offset$ ▷ 同构建时的计算方式
 - 2: 从磁盘加载文件 `partition_rank.bin` 至本地 HNSW 索引 H_{local}
-

trade-off 测试 与先前的 trade-off 测试一样，本阶段关键参数因子为：节点数 $nodes$ 和申请核心数 ppn , $np = nodes \times ppn$ 。

表 4: HNSW P 划分 MPI 优化算法综合性能对比 (最优值加粗)

nodes	ppn	np	recall	延迟 (us)	加速比
1	1	1	0.992151	587.286	1.00×
1	2	2	0.992151	491.539	1.19×
1	4	4	0.992151	662.953	0.89×
1	8	8	0.992151	433.925	1.35×
2	1	2	0.992151	462.806	1.27×
2	2	4	0.992151	579.365	1.01×
2	4	8	0.992151	520.486	1.13×
2	8	16	0.992151	553.399	1.06×
4	1	4	0.992151	437.365	1.34×
4	2	8	0.992151	662.297	0.89×
4	4	16	0.992151	583.922	1.01×
4	8	32	0.992151	655.953	0.90×

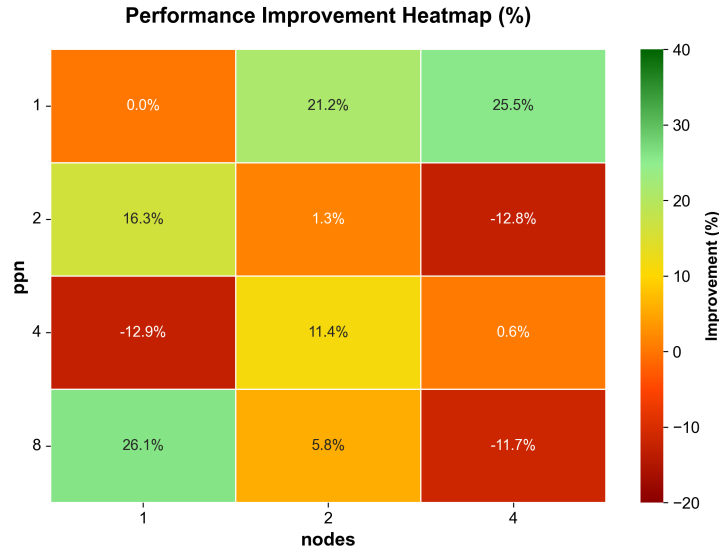


图 7: Partition-HNSW-MPI performance_heatmap

针对 Partition-HNSW-MPI, 出现了意料之外的测试结果: 算法维持了极高的 recall, 同时 nodes 和 ppn 的参数调节似乎对延迟的影响非常微小且分布无规律, 这引起了我们的思考。

经过理论分析, 这是由于算法本身与 IVF-HNSW 的底层逻辑不同所致。在 IVF-HNSW 算法中, 我们实际利用 nprobe 参数裁剪了搜索空间, 可以通过控制 nprobe 用极少的计算量换取极低的延迟。而在 Partition-HNSW 算法中, 我们并没有这一步剪枝过程, 而是对于每个分区执行了全局搜索, 这也是为什么算法的 recall 极高, 在忽略浮点精度丢失的前提下达到了与最基础串行算法 flat_scan 一致的 recall。

当我们增加进程数时, 每个进程负责的数据量从 N 降到了 N/P , 但是实际上, HNSW 的搜索时间复杂度 $O(\log N)$, 在我们当前的全局搜索前提下, 计算开销降低并不显著。同时, 需要传输的 top-k 结果数量从 p 增加到了 $P \times k$, 且所有进程并行搜索各自的 HNSW, MPI_Gatherv 同步收集所有进程的 top-k 结果到 Rank 0, 因此, 通信开销的增大实际与计算开销的降低形成了平衡。

3. HNSW ON HNSW MPI

上述 Partition-HNSW-MPI 通过数据集划分做出了 HNSW 的优化尝试, 这是一大方向。我们也可以选取其他方向, 开展对 HNSW 算法的优化探索, 如下述的 HNSW ON HNSW 算法。

HNSW ON HNSW 本质是对 IVF+HNSW 的嵌套优化, 将第一层的 IVF 倒排索引构建替换为小型的 HNSW (用聚类中心点构建) 做粗筛, 第二层依然是每个簇内的 HNSW 细查, 这样在进行 MPI 优化时, 每个进程可以直接用自己的顶层 HNSW 搜出最近的 $nprobe$ 个簇, 省去了先前 IVF+HNSW 算法中 Gather 所有距离到 Rank 0, 排序选出 $nprobe$ 个簇, 再 Bcast 给所有进程的开销。

根据上述原理, 我们设计了新的 `hnsw_on_hnsw_mpi.h`:

Algorithm 9 HNSW-on-HNSW-MPI 索引构建

Input: 基数据集 $\mathcal{B} = \{\mathbf{x}_i\}_{i=1}^n$, 维度 d , 簇数 $nlist$, 底层 HNSW 参数 $M, ef_construction$, 顶层 HNSW 参数 $coarse_M, coarse_ef_construction$, MPI 进程号 $rank$, 进程总数 $size$

Output: 磁盘上保存的索引文件

```

1: 创建索引目录 (进程 0 执行, 之后同步所有进程)
2: if  $rank = 0$  then
3:   对  $\mathcal{B}$  执行 KMeans 聚类, 得到聚类中心  $\mathbf{C} = \{\mathbf{c}_j\}_{j=1}^{nlist}$ 
4:   将  $\mathbf{C}$  保存至文件 centroids.bin
5:   构建顶层粗筛 HNSW 索引  $H_{coarse}$ , 参数为  $coarse\_M, coarse\_ef\_construction$ 
6:   for 每个聚类中心  $\mathbf{c}_j \in \mathbf{C}$  do
7:     将  $\mathbf{c}_j$  插入  $H_{coarse}$ , ID 为  $j$ 
8:   end for
9:   保存  $H_{coarse}$  至文件 coarse_hnsw.bin
10: end if
11: 广播  $\mathbf{C}$  至所有进程 ▷ MPI_Bcast
12: 计算本进程负责的簇范围  $[start\_cluster, end\_cluster)$  ▷ 同 IVF-HNSW-MPI 的划分方式
13: for 每个向量  $\mathbf{x}_i \in \mathcal{B}$  do
14:    $best\_cluster \leftarrow \arg \min_j IPDist(\mathbf{x}_i, \mathbf{c}_j)$ 
15:   将  $\mathbf{x}_i$  及其 ID 分配至簇  $best\_cluster$ 
16: end for
17: for 每个本进程负责的簇  $c \in [start\_cluster, end\_cluster)$  do
18:   if 簇  $c$  非空 then
19:     创建底层 HNSW 索引  $H_c$ , 参数为  $M, ef\_construction$ 
20:     将簇  $c$  中所有向量并行插入  $H_c$  ▷ OpenMP 并行
21:     保存  $H_c$  至文件 cluster_c.bin
22:   end if
23: end for
24: 同步所有进程 ▷ MPI_Barrier

```

Algorithm 10 HNSW-on-HNSW-MPI 索引加载

Input: 索引路径 $path$, MPI 进程号 $rank$, 进程总数 $size$

Output: 各进程加载其负责的簇索引并在内存构建顶层 HNSW

```

1: if  $rank = 0$  then
2:   从文件读取聚类中心  $\mathbf{C}$ 
3: end if

```



```

4: 广播  $C$  至所有进程 ▷ MPI_Bcast
5: 所有进程在内存中利用  $C$  重建顶层粗筛 HNSW 索引  $H_{coarse}$  ▷ 避免并发读磁盘引起的崩溃
6: 计算本进程负责的簇范围  $[start\_cluster, end\_cluster)$ 
7: for 每个簇  $c \in [start\_cluster, end\_cluster)$  do
8:   if 文件 cluster_c.bin 存在 then
9:     从磁盘加载底层 HNSW 索引  $H_c$ 
10:   end if
11: end for

```

Algorithm 11 HNSW-on-HNSW-MPI 查询搜索

Input: 查询向量 q , 返回数 k , 探测簇数 $nprobe$, 搜索参数 ef_search

Output: 全局 Top- k 最近邻结果 $\mathcal{R}$

阶段 1: 粗筛阶段 (纯本地执行, 无需 MPI 通信)

```

1: 设置  $H_{coarse}.ef \leftarrow \max(nprobe \times 2, 64)$ 
2:  $P \leftarrow H_{coarse}.searchKnn(q, nprobe)$  ▷ 在顶层 HNSW 中搜索最近的  $nprobe$  个簇

```

阶段 2: 细查阶段

```

3:  $\mathcal{R}_{local} \leftarrow \emptyset$ 
4: for 每个探测簇  $c \in P$  do
5:   if  $c \in [start\_cluster, end\_cluster)$  且  $H_c \neq \emptyset$  then
6:     设置  $H_c.ef \leftarrow ef\_search$ 
7:      $\mathcal{R}_c \leftarrow H_c.searchKnn(q, k)$ 
8:      $\mathcal{R}_{local} \leftarrow \mathcal{R}_{local} \cup \mathcal{R}_c$ 
9:   end if
10: end for

```

阶段 3: 全局结果合并

```

11: 收集所有进程的  $\mathcal{R}_{local}$  至进程 0 ▷ MPI_Gatherv
12: if  $rank = 0$  then
13:    $\mathcal{R}_{global} \leftarrow$  合并所有进程的局部搜索结果
14:   对  $\mathcal{R}_{global}$  按距离升序部分排序, 取前  $k$  个
15:    $\mathcal{R} \leftarrow$  全局 Top- $k$  结果
16: end if
17: return  $\mathcal{R}$  ▷ 仅进程 0 拥有完整结果

```

trade-off 测试 与先前的 trade-off 测试一样, 本阶段关键参数因子为: 节点数 $nodes$ 和申请核心数 ppn , $np = nodes \times ppn$.

表 5: HNSW ON HNSW MPI 优化算法综合性能对比 (最优值加粗)

nodes	ppn	np	recall	延迟 (us)	加速比
1	1	1	0.918302	1024.180	1.00×
1	2	2	0.918302	659.807	1.55×
1	4	4	0.918302	468.807	2.18×
1	8	8	0.918302	329.497	3.11×
2	1	2	0.918302	783.229	1.31×
2	2	4	0.918302	522.346	1.96×
2	4	8	0.918302	420.963	2.43×
2	8	16	0.918302	429.231	2.39×
4	1	4	0.918302	625.581	1.64×
4	2	8	0.918302	490.654	2.09×
4	4	16	0.918302	430.930	2.38×
4	8	32	0.918302	538.726	1.90×

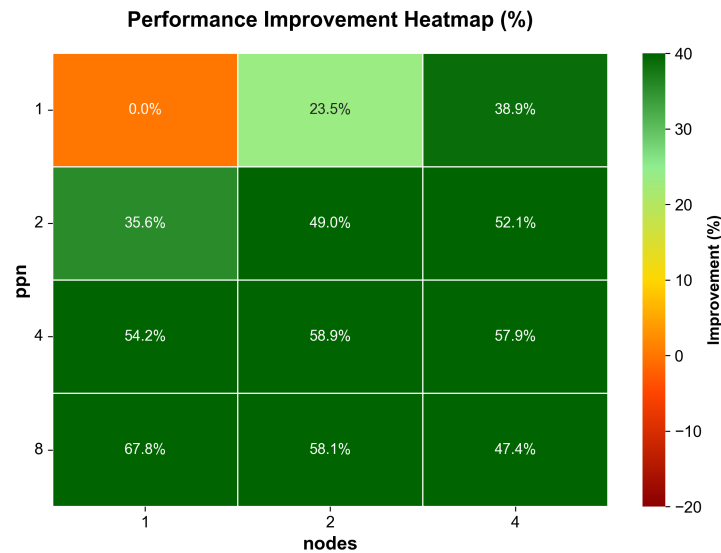


图 8: HNSW-ON-HNSW MPI performance_heatmap

相较于 IVF+HNSW MPI, HNSW ON HNSW MPI 出现了以下的新趋势:

- 同参数条件下 latency 升高: 我们消除了粗筛阶段的跨进程通信开销, 代价是顶层 HNSW 的搜索计算量显著增加, 导致粗筛阶段的本地计算延迟增加, 所以在同参数条件下, HNSW ON HNSW 表现出了比 IVF+HNSW MPI 更高的 latency。
- ppn 的优化效果更加稳定: ivf-hnsw-mpi.h 中, ppn 主要减少单节点进程数以降低内存带宽争用; 而 hnsw-on-hnsw-mpi.h 中, ppn 增加可提升顶层 HNSW 的粗筛并行性, 同时通过更细粒度的进程划分降低细查阶段的负载不均。
- 跨节点的通信开销得到缓解: nodes 从 1 增加到 4 时, 延迟从 329.497 us 上升至 625.581 us (增长约 90%), 而对于 IVF+HNSW MPI, nodes 从 1 增加到 4 时, 延迟从 259.691 us 上升至 625.581 us (增长约 141%)。hnsw-on-hnsw-mpi.h 的延迟上升幅度更小, 核心原因是粗筛阶段本地化减少了跨节点通信的累积开销, 使得 nodes 增加对性能的负面影响减弱。

五、 总结

在 MPI 编程实验中，我们探索了多种近似最近邻搜索算法在分布式环境下的并行实现与组合策略。首先，我们实现了基于 IVF 的 MPI 并行搜索，将基础数据划分到不同进程，每个进程维护各自的倒排列表，通过广播查询向量、局部搜索与全局结果合并完成检索；进一步结合 Pthread 或 OpenMP 实现混合并行，在进程内进行查询级或簇级并行优化。其次，我们尝试了图索引算法的 MPI 并行，包括 IVF+HNSW（先 IVF 聚类再在簇内建 HNSW）和基于数据集划分的 HNSW 方案（将数据随机或启发式划分为多个部分，各部分独立建图后并行搜索并合并结果），也探讨了 HNSW 嵌套（HNSW-on-HNSW）等组合策略。

通过实验，我们得出以下结论：在 MPI 并行中，负载均衡、通信开销与合并代价对性能影响显著，进程与线程数的设置需权衡启动开销与并行收益，盲目增加并行度可能导致负优化；recall 与 latency 之间的权衡始终是核心挑战，不同并行策略和算法组合会产生不同的精度 - 延迟特性。

在此过程中，我们深入学习了 MPI 多进程编程模型与进程间通信机制，理解了分布式环境下的数据划分、任务调度与结果归约方法，掌握了如何将 SIMD、多线程与 MPI 多级并行结合以提升检索效率，并进一步体会到在大规模向量检索系统中平衡计算、通信与存储约束的设计思路。

附录：AI 使用说明

本次实验中，我使用了智谱 glm5.1 进行了以下辅助分析：

- 代码调试与性能分析讨论
- 实验数据表格的 LaTeX 排版
- 关于实现代码的原理分析与交叉印证
- 关于算法瓶颈的探讨和优化思路

典型对话记录链接如下：

[对话记录 1](#)

[对话记录 2](#)

所有 AI 生成的文字和代码均经过本人充分理解与验证，符合学术诚信。